

연구 방향 1: On-Demand Layering 분석 보고서

날짜

2026-02-25

작성자

노승우

Alpamayo VRAM Optimization Research

목차 (Table of Contents)

1. On-Demand Layering 개요 (RTSS 2022)
 - 1.1 핵심 아이디어
 - 1.2 3단계 파이프라인
 - 1.3 원 논문 성과
 - 1.4 원 논문 한계
2. Alpamayo 레이어 구조 분석 결과
 - 2.1 전체 모델 구성
 - 2.2 VLM 구조 상세
 - 2.3 Expert (Trajectory Decoder) 구조 상세
 - 2.4 추론 흐름
 - 2.5 시각화 참조
3. device_map 실험 결과
 - 3.1 실험 설정
 - 3.2 디바이스 배치 결과
 - 3.3 VRAM 사용량
 - 3.4 추론 결과
 - 3.5 시각화 참조
4. 수동 오프로딩 실험 결과
 - 4.1 실험 설정
 - 4.2 디바이스 분포
 - 4.3 모듈별 VRAM 추정
 - 4.4 핵심 발견
 - 4.5 추론 결과
 - 4.6 시각화 참조
5. 적용 가능성 판단
 - 5.1 종합 판단: 제한적으로 가능 (커스텀 구현 필요)
 - 5.2 시나리오별 적용 가능성
 - 5.3 적용 불가능한 이유 (기존 프레임워크)
 - 5.4 적용 가능한 경로
6. 예상 효과 및 한계
 - 6.1 예상 효과

6.2 On-Demand Layering의 Alpamayo 특수 한계

6.3 권장 전략

7. 시각화 참조

부록: 실험 데이터 파일

연구 방향 1: On-Demand Layering 분석 보고서

실험일: 2026-02-25 환경: NVIDIA GeForce RTX 3080 Ti (12 GB VRAM), 15 GB System RAM
모델: nvidia/Alpamayo-R1-10B (11.08B params, BF16) PyTorch 2.8.0+cu128, Transformers 4.57.1, Accelerate 1.12.0

1. On-Demand Layering 개요 (RTSS 2022)

1.1 핵심 아이디어

RTSS 2022 논문 “Demand Layering for Real-Time DNN Inference with Minimized Memory Usage”는 DNN 추론 시 모델 파라미터를 전부 GPU 메모리에 로딩하지 않고, NVMe SSD를 협력 파트너로 활용하여 레이어 단위로 로딩/실행하는 기법을 제안한다.

1.2 3단계 파이프라인

```
Layer i:   [ Read_i   ][ Copy_i   ][ Kernel_i ]
Layer i+1:           [ Read_i+1 ][ Copy_i+1 ][ Kernel_i+1 ]
Layer i+2:                   [ Read_i+2 ][ Copy_i+2 ][ Kernel_i+2 ]
```

- **Read**: SSD에서 CPU 메모리로 파라미터 읽기 (DMA)
- **Copy**: CPU 메모리에서 GPU 메모리로 복사 (PCIe)
- **Kernel**: GPU에서 레이어 실행

1.3 원 논문 성과

메트릭	수치
평균 메모리 절감율	96.5%
평균 지연 오버헤드	14.8%
저지연 설정 메모리 절감율	88.4%

1.4 원 논문 한계

- 임베디드 GPU(통합 메모리)에 최적화되어 디스크리트 GPU 검증 부족
- 단일 DNN 추론 위주, LLM/VLM 미검증
- Diffusion 모델처럼 동일 레이어를 반복 호출하는 구조 미고려

2. Alpamayo 레이어 구조 분석 결과

2.1 전체 모델 구성

Alpamayo R1은 **11.08B 파라미터**, FP16/BF16 기준 **22.16 GB** 크기의 모델이다.

컴포넌트	파라미터 수	메모리 (BF16)	비율
Vision Encoder (<code>vlm.visual</code>)	576.39M	1.15 GB	5.2%
VLM Language Model (<code>vlm.model.language_model</code>)	7.58B	15.17 GB	68.5%
VLM LM Head (<code>vlm.lm_head</code>)	637.73M	1.28 GB	5.8%
Expert / Trajectory Decoder (<code>expert</code>)	2.28B	4.56 GB	20.6%
Action Projections	1.35M	2.71 MB	<0.1%
Diffusion (Flow Matching)	0 (파라미터 없음)	0	0%
합계	11.08B	22.16 GB	100%

2.2 VLM 구조 상세

VLM은 Qwen3-VL-8B 기반으로 다음과 같은 구조를 가진다:

- **Vision Encoder**: 576.39M params, 1.15 GB
- Qwen3-VL의 시각 인코더 (ViT 기반)
- **Language Model**: 36개 Transformer 레이어
- 레이어당: 192.9M params, ~386 MB (BF16)
- 전체 36 레이어: ~6.94B params, ~13.89 GB

- 임베딩 + 기타: 637.7M params, ~1.28 GB
- **LM Head**: 637.73M params, 1.28 GB

2.3 Expert (Trajectory Decoder) 구조 상세

- **36개 Transformer 레이어** (VLM의 text_config에서 파생)
- 레이어당: 63.31M params, ~126.62 MB (BF16)
- 전체 36 레이어: 2.28B params, 4.56 GB
- `embed_tokens` 는 삭제되어 VLM의 KV Cache를 재사용

2.4 추론 흐름

[입력 이미지] → Vision Encoder → VLM generate() → Expert step_fn() x 10 → [궤적 출력]
 (autoregressive) (diffusion sampling)

핵심 관찰: Expert는 VLM의 `past_key_values` (KV Cache)를 사용하며, Flow Matching으로 10회 디노이징 스텝을 수행한다.

2.5 시각화 참조

- 그림 1: `figures/01_memory_distribution.png` - 서브모듈별 메모리 분포

3. device_map 실험 결과

3.1 실험 설정

- `device_map="auto"` : Accelerate가 자동으로 GPU/CPU/Disk에 레이어 배치
- 모델: nvidia/Alpamayo-R1-10B, dtype=bfloat16
- GPU 제한: 12 GB (RTX 3080 Ti)

3.2 디바이스 배치 결과

배치 위치	파라미터 수	비율
GPU (CUDA)	4.49B	40.6%
CPU	0.00B	0.0%
Disk (meta)	6.58B	59.4%

VLM 레이어별 배치:

컴포넌트	위치
Vision Encoder (<code>vlm.model.visual</code>)	GPU
VLM 임베딩 (<code>vlm.model.language_model</code>)	GPU
VLM 레이어 0-16 (17개)	GPU
VLM 레이어 17-35 (19개)	CPU/Disk
VLM LM Head	CPU/Disk
Expert 전체 (36 레이어)	CPU/Disk
Action Projections	CPU/Disk

3.3 VRAM 사용량

메트릭	수치
모델 로드 후 VRAM	8.99 GB
피크 VRAM	9.06 GB
로드 시간	22.7초

3.4 추론 결과

추론 실패: CUDA device-side assert triggered

원인 분석: Alpamayo R1의 `fuse_traj_tokens()` 메서드에서 `masked_scatter` 연산이 사용되는데, 이 연산은 입력 텐서와 소스 텐서가 동일한 디바이스에 있어야 한다. `device_map="auto"` 로 레이어가 분산 배치되면서 커스텀 토큰 처리 로직에서 디바이스 불일치가 발생했다.

이는 중요한 발견이다: Alpamayo의 커스텀 토큰 처리 파이프라인은 단순한 `device_map` 분산 배치와 호환되지 않는다.

3.5 시각화 참조

- 그림 2: `figures/02_device_map_analysis.png` - device_map 배치 분석

4. 수동 오프로딩 실험 결과

4.1 실험 설정

시스템 RAM (15 GB)이 전체 모델 (22 GB)을 CPU에 로드하기에 부족하므로, `max_memory` 제약을 사용하여 GPU 10 GB + CPU 10 GB + Disk 오프로딩 구성으로 실험했다.

4.2 디바이스 분포

배치 위치	파라미터 수	비율
GPU	4.69B	42%
CPU	0.00B	0%
Disk	6.39B	58%

4.3 모듈별 VRAM 추정

각 서브모듈을 개별적으로 GPU에 올렸을 때의 VRAM 사용량:

단계	모듈	VRAM (BF16)	활성화 추정	합계
Phase 1	Vision Encoder	1.15 GB	~0.5 GB	~1.65 GB
Phase 2	VLM LM + Head	16.44 GB	~2.0 GB	~18.44 GB
Phase 3	Expert	4.56 GB	~1.0 GB	~5.56 GB

4.4 핵심 발견

1. **VLM Language Model이 병목**: BF16 기준 16.44 GB로 12 GB VRAM을 크게 초과
2. **모듈 단위 순차 오프로딩만으로는 불가능**: VLM 단독으로도 GPU에 올릴 수 없음
3. **레이어 단위 On-Demand Layering이 필수적**: VLM을 레이어별로 로딩해야 12 GB 내 실행 가능

4.5 추론 결과

`device_map` 방식과 동일한 이유로 **추론 실패** (커스텀 토큰 처리의 디바이스 불일치). 이는 단순 `accelerate` 기반 오프로딩이 Alpamayo의 복잡한 파이프라인과 호환되지 않음을 의미한다.

4.6 시각화 참조

- 그림 4: `figures/04_per_stage_vram.png` - 모듈별 VRAM 분석

5. 적용 가능성 판단

5.1 종합 판단: 제한적으로 가능 (커스텀 구현 필요)

On-Demand Layering의 핵심 개념(레이어별 순차 로딩/실행)은 Alpamayo에 적용 가능하나, 기존 프레임워크(`accelerate`, `device_map`)의 자동 오프로딩 기능만으로는 구현 불가능하다. 커스텀 구현이 필요하다.

5.2 시나리오별 적용 가능성

시나리오	피크 VRAM	메모리 절감	지연 오버헤드	실현성
A: 순수 레이어별 로딩 (BF16)	~1.8 GB	92%	+15-30%	높음 (구현 복잡)
B: 모듈별 순차 로딩 (BF16)	~18.4 GB	17%	모듈 전환 시간	불가능 (VLM > 12 GB)
C: INT4 양자화 + 순차 로딩 (X — 배제: 양자화 - 모델 정확도 저하 방식)	~6.1 GB	72%	+20-40%	매우 높음 (권장)

5.3 적용 불가능한 이유 (기존 프레임워크)

1. 커스텀 토큰 처리: `fuse_traj_tokens()` 의 `masked_scatter` 가 디바이스 분산과 비호환
2. KV Cache 공유: Expert가 VLM의 `past_key_values` 를 직접 참조하므로, VLM과 Expert가 별도 디바이스에 있을 수 없음
3. `generate()` 내부 로직: HuggingFace의 `generate()` 는 내부적으로 모든 모듈이 동일 디바이스에 있다고 가정
4. 시스템 RAM 제한: 15 GB RAM으로 22 GB 모델을 CPU에 전부 올릴 수 없음

5.4 적용 가능한 경로

1. INT4 양자화 우선 적용 -> 모델 크기를 ~5.5 GB로 축소 -> 12 GB 이내 전체 로딩 가능

2. **커스텀 레이어별 오프로딩 구현**: VLM의 36개 Transformer 레이어를 개별적으로 GPU에 로딩/실행하는 커스텀 forward 구현
3. **accelerate의 `dispatch_model` 커스터마이징**: 오프로딩 혹은 Alpamayo의 커스텀 토큰 처리에 맞게 수정
4. **Diffusers의 `enable_group_offload` 참고**: CUDA Stream 기반 비동기 프리페치 아이디어 적용

6. 예상 효과 및 한계

6.1 예상 효과

전략	피크 VRAM	추론 시간 (추정)	구현 난이도
FP16 Baseline	21.52 GB	273.79s	N/A (불가)
INT4 양자화만 (X — 배제)	~5.5 GB	~300-350s	낮음
On-Demand Layering (BF16)	~1.8 GB	~350-400s	매우 높음
On-Demand Layering (INT4) (X — 배제)	~0.5-1.0 GB	~300-350s	매우 높음
INT4 + 순차 모듈 오프로딩 (X — 배제)	~6.1 GB	~330-380s	중간

6.2 On-Demand Layering의 Alpamayo 특수 한계

1. **Expert의 반복 호출 문제**: - Expert는 Flow Matching으로 10회 디노이징 스텝 수행 - 각 스텝에서 36개 레이어를 모두 순회 - On-Demand Layering 적용 시 $36 \times 10 = 360$ 회 레이어 로딩 필요 - 이는 원 논문의 단일 순회(single-pass) 가정과 다름
2. **KV Cache 메모리**: - VLM generate() 시 KV Cache가 점진적으로 증가 - 레이어별 로딩을 해도 KV Cache는 GPU에 유지해야 함 - 256 토큰 생성 시 KV Cache ~1-2 GB 추가
3. **PCIe 대역폭 한계**: - PCIe 3.0 x16: ~15.75 GB/s, PCIe 4.0 x16: ~31.5 GB/s - VLM 레이어 1개 (386 MB) 전송: PCIe 4.0 기준 ~12ms - 36 레이어 전체: ~440ms (파이프라인 없이) - 파이프라인으로 은닉 가능하나, GPU 연산보다 전송이 느릴 수 있음
4. **시스템 RAM 제한**: - 현재 시스템 15 GB RAM으로는 전체 모델을 CPU에 두고 레이어별로 GPU에 전송하는 것이 불가능 - 디스크 오프로딩 필요 시 SSD I/O 지연 추가

6.3 권장 전략

최적 조합: INT4 양자화 (X — 배제: 모델 정확도 저하 방식) + 모듈 단위 순차 오프로딩 (시나리오 C)

1. Vision Encoder (INT4) -> GPU 로딩 (0.29 GB) -> 실행 -> CPU로 이동
2. VLM (INT4) -> GPU 로딩 (4.11 GB) -> generate() 실행 -> CPU로 이동
3. Expert (INT4) + Action Projections -> GPU 로딩 (1.14 GB) -> 10 스텝 실행
4. 피크 VRAM: 6.1 GB (12 GB 이내)

이 전략이 가능하려면: - INT4 양자화가 모델 정확도에 미치는 영향 검증 필요 - 커스텀 토큰 처리 로직의 디바이스 호환성 수정 필요 - 시스템 RAM이 양자화된 모델 크기(~5.5 GB)를 수용할 수 있어야 함

7. 시각화 참조

그림	파일명	설명
1	figures/01_memory_distribution.png	서브모듈별 메모리 분포 (파이 차트 + 스택 바)
2	figures/02_device_map_analysis.png	device_map="auto" 디바이스 배치 분석
3	figures/03_strategy_comparison.png	FP16 baseline vs 각 시나리오 비교
4	figures/04_per_stage_vram.png	모듈별 순차 로딩 시 VRAM (BF16 vs INT4)
5	figures/05_pipeline_diagram.png	On-Demand Layering 파이프라인 다이어그램

부록: 실험 데이터 파일

파일	설명
<code>layer_analysis.json</code>	Part 1 레이어 분석 결과
<code>device_map_results.json</code>	Part 2a device_map 실험 결과
<code>device_map_vram_timeline.csv</code>	Part 2a VRAM 시계열 데이터
<code>manual_offload_results.json</code>	Part 2b 수동 오프로딩 실험 결과
<code>manual_offload_vram_timeline.csv</code>	Part 2b VRAM 시계열 데이터
<code>analyze_layers.py</code>	Part 1 분석 스크립트
<code>test_device_map.py</code>	Part 2a 실험 스크립트
<code>test_manual_offload.py</code>	Part 2b 실험 스크립트
<code>create_figures.py</code>	Part 3 시각화 스크립트